

Dynrender: A Dynamic 3D Mesh Environment Renderer

Anish Kallu, Raunak Mondal
Course: 21-241 Matrix Algebra

December 5, 2025

Short Abstract

Dynrender is a tool that allows the user to input a JSON configuration file with object file locations and positions. Dynrender then renders the objects using OpenGL and allows the user to navigate the space with their keyboard.

1 Mathematical Background

1.1 Homogeneous Coordinates & 4-by-4 Transformation Matrices

In 3D graphics, homogeneous coordinates are used to represent 3D points as 4D vectors $[x, y, z, 1]^T$ ("Homogenous Coordinates"). For example, the homogenous coordinates $[x, y, z, 1]$ represent the same point as the homogenous coordinates $[2x, 2y, 2z, 2]$, and this still applies if you substitute 2 for any other non-zero scalar (Tomasi). This allows translations, which are not linear in 3D, to be expressed as matrix multiplications using 4×4 matrices.

The 4-by-4 matrix for translation is:

$$T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This makes sense given that multiplying the rows of T with a vector with 4 entries will result in each entry being added by the fourth entry multiplied by one of t_x, t_y, t_z , respectively (Dalling). The motivation for creating such a matrix is realizing that the original entries of a vector won't be scaled but the translation coefficients will be added, and since $[x, y, z, 1]$ is equivalent to $[cx, cy, cz, c]$ for a nonzero scalar c , the translation coefficients will need to be scaled by the fourth entry before getting added to the first 3 entries.

The 4-by-4 matrix for scaling is:

$$S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This makes sense given that multiplying the rows of T with a vector with 4 entries will result in each entry being multiplied by one of s_x, s_y, s_z , respectively. The motivation for creating such a matrix is realizing that the original entries of a vector will be scaled by such scalar coefficients but no coefficients will need to be added, and that the last entry doesn't need to be scaled. Such appropriate rotation matrices will be covered in the next section.

1.2 Rodrigues' Rotation Formula

Rodrigues's rotation formula shows that for rotation about an arbitrary axis defined by the unit vector $\mathbf{k}$ by angle θ , the rotation matrix is:

$$R = I + (\sin \theta)w + (1 - \cos \theta)w^2$$

where w is the antisymmetric cross-product matrix for k :

$$w = \begin{bmatrix} 0 & -k_z & k_y \\ k_z & 0 & -k_x \\ -k_y & k_x & 0 \end{bmatrix}$$

(Weisstein).

Note that an antisymmetric matrix is a matrix A that satisfies the property

$$A = -A^T$$

This is important because the only information that could be available to a graphics system could only be the general direction of the axis of rotation and the angle of rotation, and would make more sense to only include in a graphics system due to data constraints.

1.3 Camera Transformations & View Matrices

In the user interface of 3D rendering platforms, especially for video games and other types of simulations, the camera is continuously moved rather than the world itself (Marschner & Shirley 28). This is usually done using a view matrix, which is the inverse of the camera's transformation matrix. This makes sense because the view matrix has to perform the opposite transformations as the transformation matrix itself.

Assuming that the camera is located at position $\mathbf{e}$ such that its orientation can be set to the orthonormal vectors $\mathbf{u}, \mathbf{v}, \mathbf{w}$, which correspond to the up, right, and forward/back directions respectively, the view matrix would be equivalent to:

$$V = \begin{bmatrix} u_x & u_y & u_z & -\mathbf{u} \cdot \mathbf{e} \\ v_x & v_y & v_z & -\mathbf{v} \cdot \mathbf{e} \\ w_x & w_y & w_z & -\mathbf{w} \cdot \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

, where $u_x, u_y, u_z, v_x, v_y, v_z, w_x, w_y, w_z$ are all derived from the results of the product of scaling or rotation matrices (Cincotti). The motivation for creating such a matrix is that the 3-by-3 equivalent of V would be

$$R = \begin{bmatrix} \mathbf{u}^T \\ \mathbf{v}^T \\ \mathbf{w}^T \end{bmatrix} = \begin{bmatrix} u_x & u_y & u_z \\ v_x & v_y & v_z \\ w_x & w_y & w_z \end{bmatrix}$$

, and to calculate the entries of the fourth column of V, we would have to multiply it by the position vector and then offset the original position by making the first 3 entries negative, and therefore we derive that:

$$R\mathbf{e} = \begin{bmatrix} \mathbf{u}^T \mathbf{e} \\ \mathbf{v}^T \mathbf{e} \\ \mathbf{w}^T \mathbf{e} \end{bmatrix} = \begin{bmatrix} \mathbf{u} \cdot \mathbf{e} \\ \mathbf{v} \cdot \mathbf{e} \\ \mathbf{w} \cdot \mathbf{e} \end{bmatrix}$$

, and so,

$$t = -R\mathbf{e} = \begin{bmatrix} -\mathbf{u} \cdot \mathbf{e} \\ -\mathbf{v} \cdot \mathbf{e} \\ -\mathbf{w} \cdot \mathbf{e} \end{bmatrix}$$

, and so V is equivalent to:

$$\begin{bmatrix} u_x & u_y & u_z & -\mathbf{u} \cdot \mathbf{e} \\ v_x & v_y & v_z & -\mathbf{v} \cdot \mathbf{e} \\ w_x & w_y & w_z & -\mathbf{w} \cdot \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

1.4 Perspective Projection

Perspective projection is an important part of rendering that is also related to matrices and matrix algebra because it simulates human vision by making distant objects appear smaller (Marschner & Shirley 153). The projection matrix for a camera with an angle representing field of view fov , aspect ratio a , near plane (real-number value) n , and far plane (real-number value) f is:

$$P = \begin{bmatrix} \frac{1}{a \cdot \tan(\frac{fov}{2})} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\frac{fov}{2})} & 0 & 0 \\ 0 & 0 & \frac{-(f+n)}{f-n} & \frac{-2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

This makes sense because the half-width of the plane is $n \cdot \tan(\frac{fov}{2})$, and by dividing by n , which is needed in order to normalize the resulting vector and

make the fourth entry of it 1, we get that P_{12} would be equivalent to $1 \cdot \tan(\frac{fov}{2})$, which is the only entry different from P_{per} in the original book. After applying P , we perform perspective division (dividing by w) to obtain normalized device coordinates. The matrix P is very similar to the matrix for OpenGL, but instead in OpenGL, every input variable is replaced with its absolute value.

2 Python Code

2.1 Frameworks

In order to implement a 3D renderer in Python, we use the following package and imports:

1. PyOpenGL - interfacing layer between OpenGL and Python
2. pywavefront - handles import of .obj files
3. jsonschema - handles error handling for configuration .json files
4. numpy - handles matrix algebra
5. pygame - handles keybinds and display of renders
6. json - handles json parsing
7. math - handles basic mathematical calculations
8. collections.defaultdict - allows for default values in objectDict

2.2 Helper Functions

In order to help with debugging and the mathematical aspects of rendering, we created a couple of functions to help make the code readable. Below are the main ones that aren't just wrappers of numpy functions

rotationMatrix(theta: float, dir: ndarray) → ndarray

This function uses Rodrigues' Rotation formula to create a matrix that allows us to rotate vectors about an arbitrary axis **dir** by **theta** radians.

Pseudocode

```
d ← cross-product matrix for dir
return I + (sin(theta))d + (1 - cos(theta))(d^2)
```

normalize(a: ndarray) → ndarray

This function normalizes vector **a**.

Pseudocode

```
return a / magnitude(a)
```

transformObject(obj: pywavefront.Wavefront, transform: ndarray)
→ void

This function transforms an object **obj** by 4 by 4 transformation matrix **transform**.

Pseudocode

```
for vertex i in obj.vertices:
    (note that vertexes have 3 rows)
    j ← vertex i with 1 row appended to it
    k ← transform * j
    vertex i ← first 3 rows of k
```

rotateObject(obj, rotMat: ndarray) → void

This function rotates an object **obj** by a rotation matrix **rotMat**.

Pseudocode

```
for vertex i in obj.vertices:
    vertex i ← rotMat * vertex i
```

renderObject(obj: pywavefront.Wavefront) → void

This function renders any wavefront (.obj) object given to it.

Pseudocode

```
tell OpenGL to only draw the wireframe of the object
tell OpenGL to render the inputted points as triangles
for face in obj's faces:
    (note that each face stores the id of a vertex in obj.vertex)
    retrieve the 3 coordinates of the vertex from obj.vertex
    tell OpenGL to render the 3 coordinates as a triangle
glEnd()
```

xyz() → void

This function renders a simple xyz coordinate guide (DEBUG ONLY).

Pseudocode

```
tell OpenGL to render the inputted points as lines
set color to red
render lines from the origin 1 unit long in the x/y/z directions
set color to white
```

rotatevec(v: ndarray, theta: float, axis: ndarray) → ndarray

This function rotates **v** by **theta** radians with an axis of rotation of **axis**.

Pseudocode

```
rotMat ← rotationMatrix of theta around axis
return rotMat * v
```

2.3 Main Logic

In order to render the game, we use PyGame to create a window as follows

```
pygame.init()
display = (800, 600)
pygame.display.set_mode(display, DOUBLEBUF|OPENGL)
```

Then we tell OpenGL how to render the user's perspective. Note that we've highlighted the math in Section 1.4 but OpenGL handles the logic for us

```
glMatrixMode(GL_PROJECTION)
glLoadIdentity()
gluPerspective(45, (display[0]/display[1]), 0.1, 500.0)
```

We instantiate a couple variables to assist, including a dictionary of keybinds activated, player position, and the right facing vector. Note that none of these values matter much other than the playerPos value.

```
active = defaultdict(bool)
playerPos = np.array([0.0, 0, 40.0])
facing = np.array([0.0, 0.0, -1.0])
rightFacing = np.array([-1.0, 0.0, 0.0])
```

In order to render the field, we've created a JSON configuration schema that allows the user to dynamically adjust what exactly they want Dynrender to display. The config file can be found at *config.json* and the schema used to errorcheck the json file can be found at *dynrender.schema.json*.

```
with open("config.json") as f:
    config = json.load(f)
with open("dynrender.schema.json") as f:
    schema = json.load(f)

jsonschema.validate(instance=config, schema=schema)
```

For every object that needs to be rendered defined in the configuration file, we go through each object, import them from the selected directory, and perform the transformations and rotations requested by the user. Note that we make the rigorous assumption that `"axis_of_rotation" in item.keys() → "angle" in item.keys()` because the schema file requires that when one is present the other must be present as well. We store all of these objects in the `objectDict` dictionary.

```
for item in config["items"]:
    objectDict[item["filepath"]] = pywavefront.Wavefront(
        item["filepath"],
        collect_faces=True
    )
    transformObject(
        objectDict[item["filepath"]],
        translationMatrix(
            item["position"]["x"],
            item["position"]["y"],
            item["position"]["z"]
        )
    )
    if "axis_of_rotation" in item.keys():
        rotateObject(
            objectDict[item["filepath"]],
            rotationMatrix(
                item["angle"],
                normalize(np.array(item["axis_of_rotation"]))
            )
        )
    )
```

Now we enter the main loop of the program. Here, we set the FPS we want to use (60 fps) and handle keybinds. If a key is activated or deactivated, we change its state in `active`. We also allow the user to quit.

```
clock = pygame.time.Clock()
run = True
while run:
    clock.tick(60)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            quit()
        if event.type == pygame.KEYDOWN:
            active[event.key] = True
        if event.type == pygame.KEYUP:
            active[event.key] = False
```

Now we set the user's position. We calculate the player view vector by adding the position of the user to the vector we store for its direction it faces. We

calculate the right-facing vector using the cross-product of the facing direction with the global up vector. Then, we calculate the user's up vector by taking the cross product of the right-facing vector and the facing vector. We finally use OpenGL's built-in `gluLookAt` to set these positions properly.

```
glMatrixMode(GL_MODELVIEW)
glLoadIdentity()
lookingAt = playerPos + normalize(facing)
rightFacing = normalize(np.cross(facing, np.array([0, 1, 0])))
correctedUp = normalize(np.cross(rightFacing, facing))

lookMat = np.array([*playerPos, *lookingAt, *correctedUp])
gluLookAt(*lookMat)
```

We start the keybind handling logic here. First, let's define some smaller helper functions and constants that weren't given earlier. We calculate the amount of seconds that occur in a frame, set the rotation and movement speeds of the user, and create a step function for movement and rotation to make movement calculations easier, converting degrees / sec to degrees / step and the same for units.

```
STEPSIZE = 1.0 / 60.0
ROTATIONSPEED = 120 # degrees / sec
MOVEMENTSPEED = 8 # units / sec

def step(i): return i * STEPSIZE;
def moveStep(): return step(MOVEMENTSPEED);
def rotStep(): return step(ROTATIONSPEED);
```

Here, we iterate through every movement method for the user (WASD / LShift / RShift / UpArrow/DownArrow/LeftArrow/RightArrow) and change the player position and/or the direction the player is facing depending on the keys pressed. Note that the WASD keys are for movement the X/Z axes (forward, backward, left, right), LShift/RShift keys are for movement in the Y axis (up, down), and the UpArrow/DownArrow/LeftArrow/RightArrow keys are for changing the camera angles.

```
if active[pygame.K_a]:
    moveVec = -moveStep() * normalize(rightFacing)
    playerPos += moveVec
if active[pygame.K_d]:
    moveVec = moveStep() * normalize(rightFacing)
    playerPos += moveVec
if active[pygame.K_w]:
    moveVec = moveStep() * normalize(facing)
    playerPos += moveVec
if active[pygame.K_s]:
    moveVec = -moveStep() * normalize(facing)
```



```

        playerPos += moveVec
    if active[pygame.K_SPACE]:
        moveVec = moveStep() * normalize(correctedUp)
        playerPos += moveVec
    if active[pygame.K_LSHIFT] or active[pygame.K_RSHIFT]:
        moveVec = -moveStep() * normalize(correctedUp)
        playerPos += moveVec
    if active[pygame.K_LEFT]:
        facing = rotateVec(facing, np.deg2rad(rotStep()), correctedUp)
    if active[pygame.K_RIGHT]:
        facing = rotateVec(facing, -np.deg2rad(rotStep()), correctedUp)
    if active[pygame.K_UP]:
        facing = rotateVec(facing, np.deg2rad(rotStep()), rightFacing)
    if active[pygame.K_DOWN]:
        facing = rotateVec(facing, -np.deg2rad(rotStep()), rightFacing)

```

We prepare the screen of the application for the drawing of a new frame by calling `glClear` to clear the screen

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
```

To end the loop, we render all the objects in our `objectDict` dictionary

```

for obj in objectDict.values():
    renderObject(obj)

```

This cycle repeats 60 times a second and handles all the logic we need.

3 Screenshots

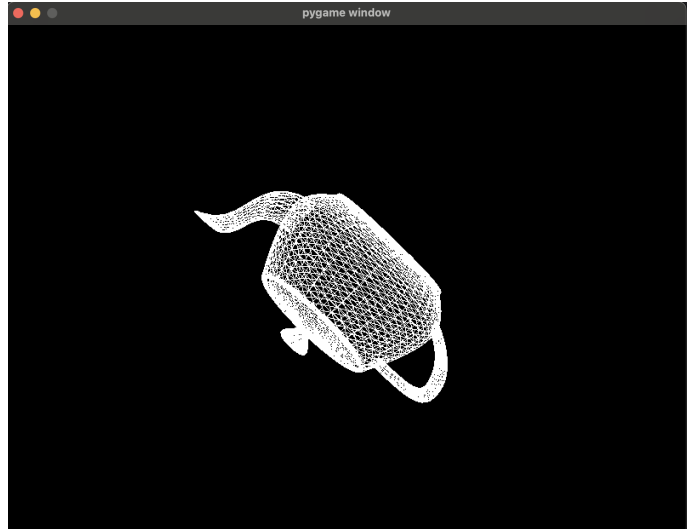


Figure 1: Rotated Utah Teapot

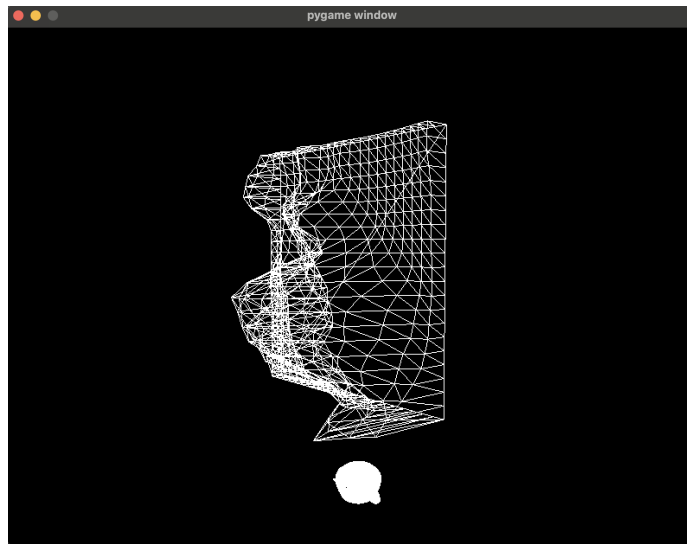


Figure 2: Simplified 3D Map of the Bay Area along with Utah Teapot

4 What we Learned

This project taught us a lot about the math behind rendering and how reliant it is on Linear Algebra. Everything from translations to rotations to perspective rendering is handled via matrix-matrix multiplication and matrix-vector multiplication. Furthermore, we learned how to use OpenGL as a tool to render objects. Overall, this project taught us a lot about the applications of Linear Algebra in the real world.

5 Appendix

5.1 dynrender.py (main code)

(Note that some code is manually reformatted to fit within the constraints of a page)

```
import json
import math

import numpy as np
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *
from collections import defaultdict
import pywavefront
import jsonschema

""" Linear Algebra Calculations """

# DEPRECATED
# project vector a onto vector b
# def proj(a, b):
#     return (a.dot(b) / b.dot(b)) * b

# DEPRECATED
# def rotationMatrix(xTheta, yTheta, zTheta):
#     xRot = np.array([[1.0, 0.0, 0.0], [0.0,
↪ math.cos(xTheta), -math.sin(xTheta)], [0.0,
↪ math.sin(xTheta), math.cos(xTheta)]])
#     yRot = np.array([[math.cos(yTheta), 0,
↪ math.sin(yTheta)], [0, 1, 0], [-math.sin(yTheta), 0,
↪ math.cos(yTheta)]])
#     zRot = np.array([[math.cos(zTheta), -math.sin(zTheta),
↪ 0], [math.sin(zTheta), math.cos(zTheta), 0], [0, 0, 1]])
#     return xRot, yRot, zRot

# DEPRECATED
# def rotationMatrix(theta, dir):
#     # Note that Y & Z are flipped due to the fact that the
↪ Y and Z axes in math & OpenGL are flipped
#     if dir == Direction.X: return np.array([[1.0, 0.0,
↪ 0.0], [0.0, math.cos(theta), -math.sin(theta)], [0.0,
↪ math.sin(theta), math.cos(theta)]])
```

```

#     if dir == Direction.Y: return
↪ np.array([[math.cos(theta), -math.sin(theta), 0],
↪ [math.sin(theta), math.cos(theta), 0], [0, 0, 1]])
#     if dir == Direction.Z: return
↪ np.array([[math.cos(theta), 0, math.sin(theta)], [0, 1,
↪ 0], [-math.sin(theta), 0, math.cos(theta)]])
#     return np.zeros((3, 3))

def rotationMatrix(theta, dir):
    # calculate cross product matrix of dir
    cpm = np.array([[0, -dir[2], dir[1]], [dir[2], 0,
↪ -dir[0]], [-dir[1], dir[0], 0]])
    return np.identity(3) + (math.sin(theta) * cpm) + ((1 -
↪ math.cos(theta)) * np.matmul(cpm, cpm))

def rotateVec(v, theta, axis):
    rotMat = rotationMatrix(theta, axis)
    return np.matmul(rotMat, v)

def rotateVecMat(v, rotMat):
    return np.matmul(rotMat, v)

def normalize(a):
    return a / np.linalg.norm(a)

def translationMatrix(x, y, z):
    return np.array([[1, 0, 0, x], [0, 1, 0, y], [0, 0, 1,
↪ z], [0, 0, 0, 1]])

# DEPRECATED
# def translate(mat, playerPos):
#     # glTranslatef(*mat)
#     playerPos[0] -= mat[0]
#     playerPos[1] -= mat[1]
#     playerPos[2] -= mat[2]

""" RENDERING """
def red():
    glColor3fv((1, 0, 0))

def green():
    glColor3fv((0, 1, 0))

def white():
    glColor3fv((1, 1, 1))

```

```

def transformObject(obj, transform):
    for i in range(len(obj.vertices)):
        npVertex = np.array([
            obj.vertices[i][0],
            obj.vertices[i][1],
            obj.vertices[i][2], 1
        ])
        # print(npVertex)
        obj.vertices[i] = np.matmul(transform,
            ↪ npVertex)[:3].tolist()
        # print(np.matmul(transform, npVertex))

def rotateObject(obj, rotMat):
    for i in range(len(obj.vertices)):
        obj.vertices[i] = rotateVecMat(obj.vertices[i],
            ↪ rotMat)

def renderObject(obj: pywavefront.Wavefront):
    glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)
    glBegin(GL_TRIANGLES)
    for v in obj.parser.mesh.faces:
        glVertex3fv(obj.vertices[v[0]])
        glVertex3fv(obj.vertices[v[1]])
        glVertex3fv(obj.vertices[v[2]])
    glEnd()
    pass

def xyz():
    glBegin(GL_LINES)
    red()
    glVertex3fv((0, 0, 0))
    glVertex3fv((1, 0, 0))
    glVertex3fv((0, 0, 0))
    glVertex3fv((0, 1, 0))
    glVertex3fv((0, 0, 0))
    glVertex3fv((0, 0, 1))
    white()
    glEnd()

def renderDebugVec(vs, ve, color = (0, 1, 0)):
    glBegin(GL_LINES)
    glColor3fv(color)
    glVertex3fv((vs[0], vs[1], vs[2]))
    glVertex3fv((ve[0], ve[1], ve[2]))
    white()
    glEnd()

```

```

STEPSIZE = 1.0 / 60.0
ROTATIONSPEED = 120 # degrees / sec
MOVEMENTSPEED = 8 # units / sec

# DEPRECATED
# class Direction(Enum):
#     X = 0
#     Y = 1
#     Z = 2

objectDict = {}

def step(i): return i * STEPSIZE;
def moveStep(): return step(MOVEMENTSPEED);
def rotStep(): return step(ROTATIONSPEED);

def main():
    pygame.init()
    display = (800, 600)
    pygame.display.set_mode(display, DOUBLEBUF|OPENGL)

    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    gluPerspective(45, (display[0]/display[1]), 0.1, 500.0)

    # flags & states
    active = defaultdict(bool)

    playerPos = np.array([0.0, 0, 40.0])
    facing = np.array([0.0, 0.0, -1.0])
    rightFacing = np.array([-1.0, 0.0, 0.0])

    # parsing logic
    with open("config.json") as f:
        config = json.load(f)
    with open("dynrender.schema.json") as f:
        schema = json.load(f)

    jsonschema.validate(instance=config, schema=schema)

    for item in config["items"]:
        objectDict[item["filepath"]] = pywavefront.Wavefront(
            item["filepath"],
            collect_faces=True
        )

```

```

transformObject(
    objectDict[item["filepath"]],
    translationMatrix(
        item["position"]["x"],
        item["position"]["y"],
        item["position"]["z"]
    )
)
if "axis_of_rotation" in item.keys():
    rotateObject(
        objectDict[item["filepath"]],
        rotationMatrix(
            item["angle"],
            normalize(np.array(item["axis_of_rotation"]))
        )
    )

clock = pygame.time.Clock()
run = True
while run:
    clock.tick(60)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            quit()
        if event.type == pygame.KEYDOWN:
            active[event.key] = True
        if event.type == pygame.KEYUP:
            active[event.key] = False

    # lrDiff = pygame.mouse.get_pos()[0] -
    ↪ mouseInitPos[0]
    # udDiff = pygame.mouse.get_pos()[1] -
    ↪ mouseInitPos[1]
    # if (lrDiff != 0 or udDiff != 0): print(lrDiff,
    ↪ udDiff)

    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()
    lookingAt = playerPos + normalize(facing)
    rightFacing = normalize(np.cross(facing, np.array([0,
    ↪ 1, 0])))
    correctedUp = normalize(np.cross(rightFacing,
    ↪ facing))

```



```

lookMat = np.array([*playerPos, *lookingAt,
    ↪ *correctedUp])
gluLookAt(*lookMat)

if active[pygame.K_a]:
    moveVec = -moveStep() * normalize(rightFacing)
    playerPos += moveVec
if active[pygame.K_d]:
    moveVec = moveStep() * normalize(rightFacing)
    playerPos += moveVec
if active[pygame.K_w]:
    moveVec = moveStep() * normalize(facing)
    playerPos += moveVec
if active[pygame.K_s]:
    moveVec = -moveStep() * normalize(facing)
    playerPos += moveVec
if active[pygame.K_SPACE]:
    moveVec = moveStep() * normalize(correctedUp)
    playerPos += moveVec
if active[pygame.K_LSHIFT] or
    ↪ active[pygame.K_RSHIFT]:
    moveVec = -moveStep() * normalize(correctedUp)
    playerPos += moveVec
if active[pygame.K_LEFT]:
    facing = rotateVec(facing, np.deg2rad(rotStep()),
    ↪ correctedUp)
if active[pygame.K_RIGHT]:
    facing = rotateVec(facing,
    ↪ -np.deg2rad(rotStep()), correctedUp)
if active[pygame.K_UP]:
    facing = rotateVec(facing, np.deg2rad(rotStep()),
    ↪ rightFacing)
if active[pygame.K_DOWN]:
    facing = rotateVec(facing,
    ↪ -np.deg2rad(rotStep()), rightFacing)
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

for obj in objectDict.values():
    renderObject(obj)

# DEBUG
# xyz()
""" renderDebugVec(
    lookingAt + (facing * 12),
    lookingAt + (facing * 12) +
    ↪ normalize(rightFacing),

```

```

        (0, 0, 1)
    )"""
    """ renderDebugVec(
        lookingAt + (facing * 12),
        lookingAt + (facing * 12) +
        ↪ normalize(correctedUp),
        (0, 1, 0)
    )"""
    pygame.display.flip()

if __name__ == '__main__':
    main()

```

5.2 dynrender.schema.json

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "dynrender configuration json schema",
  "type": "object",
  "properties": {
    "items": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "filepath": {
            "type": "string"
          },
          "position": {
            "type": "object",
            "properties": {
              "x": {
                "type": "number"
              },
              "y": {
                "type": "number"
              },
              "z": {
                "type": "number"
              }
            }
          },
          "required": [
            "x",
            "y",
            "z"
          ]
        }
      }
    }
  },

```

```

        "axis_of_rotation": {
            "type": "array",
            "items": {
                "type": "number"
            }
        },
        "angle": {
            "type": "number"
        }
    },
    "required": [
        "filepath",
        "position"
    ],
    "dependentRequired": {
        "axis_of_rotation": ["angle"],
        "angle": ["axis_of_rotation"]
    }
}
},
"required": [
    "items"
]
}

```

Bibliography

3D projection - Wikipedia, https://en.wikipedia.org/wiki/3D_projection. Accessed 3 December 2025.

Ahn, Song Ho. "OpenGL Projection Matrix." *songho.ca*,
https://www.songho.ca/opengl/gl_projectionmatrix.html. Accessed 3 December 2025.

Cincotti, Carmen. "The Magic of the LookAt Matrix. Linear Algebra is magical. Implementing... | by Carmen Cincotti." *Medium*, 25 April 2022,
<https://medium.com/@carmencincotti/lets-look-at-magic-lookat-matrices-c77e53ebdf78>.
Accessed 4 December 2025.

Dalling, Tom. "Explaining Homogeneous Coordinates & Projective Geometry — Tom Dalling." *Tom Dalling*, 24 February 2014,
<https://www.tomdalling.com/blog/modern-opengl/explaining-homogenous-coordinates-and-projective-geometry/>. Accessed 3 December 2025.

"Homogeneous Coordinates." *Wikipedia*, 21 Jan. 2021,
en.wikipedia.org/wiki/Homogeneous_coordinates.

Marschner, Steve, and Peter Shirley. *Fundamentals of Computer Graphics, Fourth Edition*.
Taylor & Francis, 2016. Accessed 3 December 2025.

Tomasi, Carlo. "Perspective Projection in Homogeneous Coordinates." *Duke Computer Science*,
<https://courses.cs.duke.edu/cps274/fall15/notes/homogeneous-coordinates.pdf>. Accessed
3 December 2025.

Transformation matrix - Wikipedia, https://en.wikipedia.org/wiki/Transformation_matrix.
Accessed 3 December 2025.

Weisstein, Eric W. "Rodrigues' Rotation Formula." *Mathworld.wolfram.com*,
mathworld.wolfram.com/RodriguesRotationFormula.html.